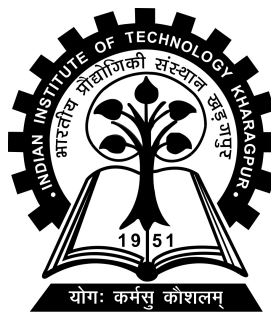


Logical Specification Guided Reinforcement Learning leveraging Reward Machines

Project-I (CS47005) report submitted to
Indian Institute of Technology Kharagpur
in partial fulfillment for the award of the degree of
Bachelor of Technology
in
Computer Science and Engineering

by
Meduri Harshith Chowdary
(21CS10042)

Under the supervision of
Dr. Aritra Hazra



Department of CSE
Indian Institute of Technology Kharagpur
Autumn Semester, 2024-25
November 5, 2024

DECLARATION

I certify that

- (a) The work contained in this report has been done by me under the guidance of my supervisor.
- (b) The work has not been submitted to any other Institute for any degree or diploma.
- (c) I have conformed to the norms and guidelines given in the Ethical Code of Conduct of the Institute.
- (d) Whenever I have used materials (data, theoretical analysis, figures, and text) from other sources, I have given due credit to them by citing them in the text of the thesis and giving their details in the references. Further, I have taken permission from the copyright owners of the sources, whenever necessary.

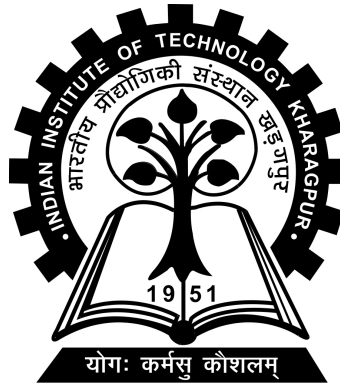
Date: November 5, 2024

Place: Kharagpur

(Meduri Harshith Chowdary)

(21CS10042)

DEPARTMENT OF CSE
INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR
KHARAGPUR - 721302, INDIA



CERTIFICATE

This is to certify that the project report entitled “Logical Specification Guided Reinforcement Learning leveraging Reward Machines” submitted by Meduri Harshith Chowdary (Roll No. 21CS10042) to Indian Institute of Technology Kharagpur towards partial fulfillment of requirements for the award of degree of Bachelor of Technology in Computer Science and Engineering is a record of bona fide work carried out by him under my supervision and guidance during Autumn Semester, 2024-25.

Date: November 5, 2024
Place: Kharagpur

Dr. Aritra Hazra
Department of CSE
Indian Institute of Technology Kharagpur
Kharagpur - 721302, India

Abstract

Reinforcement Learning (RL) has emerged as a powerful tool for developing adaptive agents capable of learning optimal policies through trial-and-error interactions with their environment. However, traditional RL approaches often face significant challenges when tasked with encoding complex, long-horizon tasks due to sparse and difficult-to-specify reward functions. In this work, we propose the use of reward machines (RMs) as a structured, automata-based representation of reward functions, which expose the underlying structure of complex rewards and facilitate more efficient policy learning. By integrating formal languages such as Linear Temporal Logic (LTL) into RL frameworks, reward functions that would otherwise be arduous to specify can be automatically translated into RMs, simplifying the development process and reducing the potential for error.

The exposed structure of RMs can be leveraged by specialized Q-learning algorithms and automated reward shaping techniques to significantly enhance the sample efficiency of RL. These RM-enhanced approaches enable agents to better interpret task requirements and align learning behavior accordingly. Our experiments demonstrate that RL methods tailored with RM guidance outperform state-of-the-art (deep) RL algorithms, solving complex problems that remain impractical for conventional techniques. This work bridges the gap between theoretical constructs and practical RL implementations, contributing to the reliability and effectiveness of agents trained for complex, real-world scenarios.

Contents

Declaration	i
Certificate	ii
Abstract	iii
Contents	iv
List of Figures	vi
1 Introduction	1
1.1 Background	2
1.2 Motivation	3
1.3 Problem Statement	3
1.4 Objectives	4
2 Literature Survey	5
2.1 Reinforcement Learning and Reward Specification Challenges	5
2.2 Vocabulary and Labeling Functions for Reward Specification	6
2.3 The Need for Non-Markovian Reward Functions	6
2.4 Non-Markovian Reward Decision Processes (NMRDPs)	8
2.5 Automata-Based Approaches to Reward Specification	8
2.6 Translating Formal Languages to Reward Machines	9
2.7 Advantages of Reward Machines	9
2.8 Recent Works	10
2.8.1 Integrating LTL with RL	10
2.8.2 Compositional RL with Logical Specifications	11
2.8.3 Significance and Future Directions	11
2.9 Summary	12
3 Methodology	13
3.1 Defining Reward Machines (RMs)	13
3.2 Constructing Reward Machines from Temporal Logic	14

3.2.1	Step 1: Transforming a Formula into a DFA	14
3.2.2	Step 2: Constructing the RM from the DFA	14
3.3	Formal Languages and Temporal Logics	15
3.4	Inducing Reward Functions Using RMs	15
3.5	Constructing RMs from Reward Specifications	15
3.6	Q-Learning for Reward Machines (QRM)	17
3.6.1	Augmenting QRM with Reward Shaping	17
3.7	Reward Shaping with Value Iteration	18
3.8	Value Iteration for Potential Functions	19
3.9	Example Application: Office World Task	20
4	Experimental Results and Observations	21
4.1	Experimental Setup	21
4.2	Environments and Tasks	22
4.3	Experimental Parameters	23
4.4	Results	23
4.5	Observations	24
5	Conclusion and Future Work	26
5.1	Summary of Work	26
5.2	Future Work	26
5.2.1	Policy Robustness to Environmental Changes	26
5.2.2	Balancing Safety Constraints: Flexibility	27
5.2.3	Scope for Future Work: Quadrant Analysis	28
5.2.4	Potential Research Directions	29

List of Figures

2.1	Mincecraft Grid World	7
4.1	Comparison of RL Approaches Across Different Environments. Each subfigure represents the performance of the Q-learning agent, QRM-based agent, and QRM-based agent with Reward Shaping (QRM + RS) in the respective environment. The performance metrics are averaged over 30 runs, with the shaded areas indicating the 25th and 75th percentiles.	24
5.1	Quadrant Analysis for Future Work in RL	28

Chapter 1

Introduction

Reinforcement Learning (RL) is a framework where an agent learns optimal behavior through exploratory interactions with an environment, aiming to maximize expected cumulative rewards. RL is particularly beneficial in complex, unmodeled environments that challenge traditional decision-making techniques. Despite its strengths, RL faces significant issues in practical applications, especially in reward specification and sample complexity. Crafting a high-fidelity reward function requires detailed consideration of both the task and the environment, and as tasks grow more complex and temporally extended, designing such functions becomes increasingly difficult. Moreover, non-trivial reward functions often deliver rewards sparsely, necessitating extensive exploratory episodes for the agent to converge to an effective policy. Formal methods, particularly logical specifications such as Linear Temporal Logic (LTL), have been effective in planning and control synthesis for long-horizon tasks. Integrating these logical constructs into RL frameworks can address the limitations of traditional reward-based approaches.

1.1 Background

In Reinforcement Learning (RL), an agent interacts with an environment to learn how to make decisions that maximize its expected cumulative reward Sutton and Barto (2018). The environment is typically represented as a Markov Decision Process (MDP), defined by the tuple $M = \langle S, A, s_0, T, r, \gamma \rangle$. Here, S is a finite set of states, A is a finite set of actions, $s_0 \in S$ is the initial state, $T(s_{t+1}|s_t, a_t)$ is the transition probability distribution, $r : S \times A \times S \rightarrow \mathbb{R}$ is the reward function, and $\gamma \in (0, 1]$ is the discount factor. At each time step t , the agent selects an action a_t , leading to a transition from state s_t to s_{t+1} according to T , and receives a reward $r(s_t, a_t, s_{t+1})$. The goal of the agent is to learn a policy $\pi(a|s)$ that maximizes the expected discounted future reward.

Tabular Q-learning Watkins and Dayan (1992) is a foundational RL algorithm that approximates the optimal Q-function $q^*(s, a)$, defined by the Bellman equation:

$$q^*(s, a) = \sum_{s' \in S} T(s'|s, a) \left[r(s, a, s') + \gamma \max_{a' \in A} q^*(s', a') \right] \quad (1.1)$$

An optimal policy π^* can be derived from this Q-function. The Q-function approximation $\tilde{q}(s, a)$ is updated using the rule:

$$\tilde{q}(s, a) \leftarrow \tilde{q}(s, a) + \alpha \left[r(s, a, s') + \gamma \max_{a'} \tilde{q}(s', a') - \tilde{q}(s, a) \right] \quad (1.2)$$

where α is the learning rate. Although tabular Q-learning converges to an optimal policy given sufficient exploration, it becomes impractical for large or continuous state spaces.

Deep Q-Networks (DQN) Mnih et al. (2015) extend Q-learning by using deep neural networks to approximate the Q-function. Unlike tabular Q-learning, DQN can handle high-dimensional input spaces but may not guarantee convergence and can

suffer from instabilities such as delusional bias Lu et al. (2018). Enhancements like Double DQN Hasselt et al. (2016) and Prioritized Experience Replay Schaul et al. (2015) have been used to improve the stability and performance of DQN, as seen in studies by Toro Icarte et al. Icarte et al. (2018b).

1.2 Motivation

Two primary challenges in RL systems are:

1. the difficulty of specifying complex reward functions
2. the high sample complexity that results in prolonged training periods.

The design of reward functions that capture multi-faceted, temporally extended tasks is a daunting process that often results in inefficient exploration and lengthy convergence times. Recent advancements by researchers like Toro Icarte et al. Icarte et al. (2018a,b) have addressed these challenges by exploiting the inherent structure of the reward function, using it as input to specialized RL algorithms. This approach significantly improves sample efficiency and accelerates convergence to high-quality policies.

1.3 Problem Statement

Traditional RL systems treat reward functions as black boxes, which limits their efficiency in learning complex tasks. The core problem lies in how to design and utilize reward functions that encode complex behaviors without resorting to costly exploratory episodes. While specific formal languages like Linear Temporal Logic (LTL) have been used to express temporally extended tasks, no single language can cover the diverse range of potential RL applications. Developing tailored learning

algorithms for each type of language is impractical and resource-intensive.

This project seeks to address these issues by introducing reward machines (RMs) as a universal, automata-based representation for reward functions. RMs capture the underlying structure of reward functions, making it possible to use a single type of specialized Q-learning algorithm for various specification languages, including LTL and regular expressions. By translating these languages into RMs, we enable RL agents to exploit the inherent structure of the reward function, improving sample efficiency and convergence rates.

1.4 Objectives

Inspired by Toro Icarte et al.’s work, this project aims to use reward machines (RMs) as a universal representation for RL reward functions. The goal is to leverage the structure of RMs to:

1. Represent reward functions as automata-based models that expose underlying task structures.
2. Demonstrate that various specification languages, including LTL and regular expressions, can be translated into RMs.
3. Utilize this representation to apply specialized Q-learning algorithms and reward-shaping techniques to improve sample efficiency.
4. Achieve superior performance in tasks that standard tabular and deep RL methods struggle to solve.

Chapter 2

Literature Survey

2.1 Reinforcement Learning and Reward Specification Challenges

In RL, an agent interacts with an unknown environment to learn how to act in ways that maximize its expected cumulative reward Sutton and Barto (2018). The environment is typically modeled as a Markov Decision Process (MDP), defined by the tuple $M = \langle S, A, s_0, T, r, \gamma \rangle$. While MDPs provide a foundational framework, the specification of reward functions poses significant challenges. An RL agent cannot inherently perceive rewards; a programmer must design the reward function, whether the environment is derived from a simulator or real-world observations. Crafting effective reward functions is often difficult for two main reasons:

1. **State Representation Limitations:** The state representation may not support direct reward specification, especially in high-dimensional or low-abstraction settings such as pixel-based inputs.
2. **Temporal Dependencies:** Rewards are Markovian by definition, mapping states (or states and actions) to scalar values. However, many tasks involve

temporally extended behaviors that require multi-step sequences, making them hard to capture with traditional Markovian reward functions.

2.2 Vocabulary and Labeling Functions for Reward Specification

A suitable vocabulary for specifying rewards is needed to address the issue of state representation. This vocabulary can be derived from state-defining variables or features extracted from states using property detectors.

For this survey, a vocabulary is defined as a set P of propositional symbols, and a labeling function $L : S \times A \times S \rightarrow 2^P$ relates experiences to truth assignments over P . This labeling can refer to events that involve state transitions and actions, allowing reward functions to account for non-Markovian aspects of RL environments.

2.3 The Need for Non-Markovian Reward Functions

Consider a Minecraft-inspired 2D grid world adapted from Andreas et al. (2017). In this environment, an agent named Luigi can collect raw materials such as wood, grass, iron, gold, and gems, and interact with structures like factories, toolsheds, and workbenches to create items like ropes or bridges. Dangers such as zombies also exist, adding complexity to task completion. Rewards are specified using a set P of propositional symbols (e.g., *got wood*, *used factory*), with a labeling function that identifies when these propositions are true.

Illustrative examples highlight the complexity of designing reward functions:



Figure 2.1: Mincecraft Grid World

- **Example E1:** “Make a bridge by collecting wood and iron in any order, and then use the factory.”
- **Example E2:** “If it’s night time, stay in the shed until daylight.”
- **Example E3:** “Always avoid zombies.”
- **Example E4:** “While there are gems on the ground, collect them and place them in your bag. When the bag is full, deliver it to the shed and get an empty bag.”

These examples illustrate that Markovian reward functions struggle to capture temporally extended, conditional behaviors. To define such non-trivial tasks, reward functions need to incorporate memory and state history, leading to the concept of Non-Markovian Reward Decision Processes (NMRDPs) Bacchus et al. (1996).

2.4 Non-Markovian Reward Decision Processes (NM-RDPs)

An NMRDP extends the standard MDP definition by allowing rewards to depend on sequences of state-action pairs. Formally, an NMRDP is a tuple $\langle S, A, s_0, T, R, \gamma \rangle$, where $R : (S \times A)^+ \times S \rightarrow \mathbb{R}$ is a non-Markovian reward function mapping finite state-action histories to scalar values. The optimal strategies for NMRDPs aim to maximize expected discounted cumulative non-Markovian rewards along a trajectory:

$$\sum_{i=0}^n \gamma^i R(h_i, s_{i+1}), \quad \text{where } h_i = (s_0, a_0) \cdots (s_i, a_i) \quad (2.1)$$

These strategies differ from MDP policies as they rely on state-action histories rather than memoryless state-action mappings.

2.5 Automata-Based Approaches to Reward Specification

Automata-like structures can be employed to represent non-Markovian rewards effectively. Reward machines (RMs) decompose non-Markovian reward functions into a Markovian form by leveraging the connection between regular languages and automata. This decomposition allows RMs to serve as a common representation for various formal languages that describe non-Markovian behavior. Such languages include:

- **Linear Temporal Logic (LTL):** Suitable for expressing temporally extended properties Emerson (1990).
- **Regular Expressions:** Capture looping and sequential behaviors Thompson (1968).

- **Other Formal Specification Languages:** Enable concise, expressive task representation Hopcroft and Ullman (1979).

2.6 Translating Formal Languages to Reward Machines

Reward machines can be constructed by translating formal language specifications into automata that map sequences of experiences to reward values. This translation leverages existing automata-compilation methods, enabling the automated construction of RMs Duret-Lutz et al. (2016). The RM structure facilitates the decomposition of complex reward functions into simpler, state-based sub-rewards, which specialized RL algorithms can then use for efficient learning and policy formation.

2.7 Advantages of Reward Machines

Reward machines offer a standardized, automata-based normal form for representing non-Markovian reward functions. This approach has several advantages:

- **Expressiveness:** Captures conditional, temporally extended, and looping behaviors.
- **Algorithm Compatibility:** Works seamlessly with existing RL algorithms tailored to exploit reward structures.
- **Language Versatility:** Translates various specification languages into a unified representation, accommodating diverse task requirements.

2.8 Recent Works

Recent advancements in reinforcement learning (RL) involving formal specifications and automata-based structures have been significantly influenced by the works of Hosein Hasanbeig (University of Oxford) and Kishore Jyotimurugan (University of Pennsylvania).

2.8.1 Integrating LTL with RL

Hosein Hasanbeig’s work has pioneered approaches where LTL specifications guide RL agents to learn policies that satisfy given temporal properties. Key contributions include:

- **Safe RL with LTL:** Hasanbeig introduced methodologies where the agent’s exploration is constrained by safety conditions encoded in LTL. This approach ensures that the agent avoids hazardous states while maximizing cumulative rewards.
- **LTL-Constrained Policy Synthesis:** By transforming LTL specifications into automata structures, Hasanbeig’s work facilitates policy synthesis where agents learn optimal strategies that satisfy temporal properties over infinite state traces.
- **Integration with Automata Learning:** His research has demonstrated how combining automata theory with RL can improve sample efficiency by providing structured guidance and decomposing complex reward signals into manageable sub-goals.

These advancements have laid the groundwork for using formal methods in RL for real-world applications, such as autonomous navigation and robotic control, where safety and adherence to specified behaviors are critical.

2.8.2 Compositional RL with Logical Specifications

Kishore Jyotimurugan’s research at UPenn has similarly delved into RL under formal specifications, focusing on the synthesis of controllers that comply with high-level task specifications expressed in temporal logics. Notable aspects of his work include:

- **Compositional RL with Logical Specifications:** Jyotimurugan has explored methods where complex task specifications are decomposed into sub-tasks, each expressed in temporal logic. This compositional approach simplifies the synthesis of RL policies that align with overarching objectives.
- **Automata-Based Task Decomposition:** His work emphasizes translating LTL and similar formal specifications into automata, enabling structured reward functions that guide RL agents more effectively through complex task sequences.
- **Synthesis Under Uncertainty:** One of the pivotal aspects of Jyotimurugan’s research has been the synthesis of policies that not only meet logical specifications but also consider uncertainty in the environment. This has practical implications for scenarios such as automated planning and multi-agent systems.

Both researchers’ works align with the ongoing trend of utilizing formal languages, automata theory, and structured reward machines in RL. Their contributions have paved the way for developing RL algorithms that leverage the structure of logical specifications, thereby improving sample efficiency and enabling the resolution of complex, temporally extended tasks.

2.8.3 Significance and Future Directions

The contributions by Hasanbeig and Jyotimurugan provide a robust framework for integrating LTL and other temporal logics with RL. Their methods ensure that

agents learn policies that are not only optimal in terms of rewards but also compliant with specified safety and task requirements. Future research may expand on these foundations by exploring:

- **Scalability to Large State Spaces:** Adapting these methods to handle larger, more complex state spaces.
- **Real-Time Implementation:** Applying these theoretical advances in real-world, real-time RL applications.
- **Hybrid Approaches:** Combining classical RL algorithms with formal methods to further enhance learning efficiency and robustness.

In summary, the works of Hasanbeig and Jyotimurugan significantly contribute to the field by demonstrating how RL can be enriched with formal specification methods to create safe, effective, and interpretable policies.

2.9 Summary

The use of reward machines addresses the inherent limitations of traditional Markovian reward functions by providing a structured approach to modeling non-Markovian rewards. By decomposing complex behaviors into manageable sub-tasks, RMs enhance the specification and learning of high-level, temporally extended tasks in RL. This methodology supports the development of more expressive, efficient, and versatile RL systems capable of solving intricate problems that standard MDPs struggle to handle.

Chapter 3

Methodology

3.1 Defining Reward Machines (RMs)

A reward machine (RM) is an automata-based representation used to express non-Markovian reward functions in reinforcement learning. Formally, an RM can be viewed as a Mealy machine $\langle Q, q_0, \Sigma, R, \delta, \rho \rangle$, where:

- Q is a finite set of states.
- $q_0 \in Q$ is the initial state.
- Σ is the input alphabet, which is 2^P (the power set of the vocabulary P).
- R is a finite set of reward functions mapping $S \times A \times S \rightarrow \mathbb{R}$.
- $\delta : Q \times \Sigma \rightarrow Q$ is the transition function.
- $\rho : Q \times \Sigma \rightarrow R$ is the output function.

The RM processes sequences of state labels and transitions through its states, determining the reward based on the current input and RM state. The induced non-Markovian reward function R assigns rewards according to the experiences the agent has and the state in the RM.

3.2 Constructing Reward Machines from Temporal Logic

A DFA processes an input string $\sigma_0\sigma_1\ldots\sigma_{n-1} \in \Sigma^*$ and accepts it if the final state q_n after processing the input satisfies $q_n \in F$. This property allows DFAs to recognize all regular languages, which is crucial for constructing reward machines from formal language specifications.

The process of constructing an RM begins with defining a formal reward specification using a temporal logic formula. The steps include:

3.2.1 Step 1: Transforming a Formula into a DFA

Temporal logic formulas, such as those expressed in Linear Temporal Logic (LTL), can be translated into DFAs. The DFA accepts input strings that satisfy the conditions specified by the formula. Each state in the DFA represents a configuration of the logical formula, and the transition function δ determines how the automaton moves between states based on the input.

3.2.2 Step 2: Constructing the RM from the DFA

Define the RM $M_R = \langle Q, q_0, \Sigma, R, \delta, \rho \rangle$ based on the DFA. The RM states Q are formed as the Cartesian product of the DFA states, and the output function ρ assigns a reward based on whether the DFA state is accepting or not.

The use of DFAs as an intermediary step ensures that any formal language, whose formulas can be expressed as a regular language, can be effectively transformed into an RM. This property leverages the well-known equivalence between regular languages and DFAs, enabling the construction of RMs that provide rich, structured reward signals for complex, temporally extended tasks.

3.3 Formal Languages and Temporal Logics

Linear Temporal Logic (LTL) is used to specify temporally extended conditions. LTL extends propositional logic with operators such as:

- $\bigcirc\alpha$ (next): α must hold in the next state.
- $\alpha\mathcal{U}\beta$ (until): α must hold until β holds.
- $\Box\alpha$ (always): α must hold at every state.
- $\Diamond\alpha$ (eventually): α must hold at some state in the future.

Example: To specify the task “Make a bridge by collecting wood and iron in any order, and using the factory afterwards,” the LTL formula would be:

$$\Diamond(\text{got wood} \wedge \Diamond\text{used factory}) \wedge \Diamond(\text{got iron} \wedge \Diamond\text{used factory})$$

3.4 Inducing Reward Functions Using RMs

The non-Markovian reward function induced by an RM is defined based on the transitions and outputs of the RM. The reward assigned to a sequence of experiences is determined using the output function ρ and the transition function δ .

3.5 Constructing RMs from Reward Specifications

Given a formal reward specification $R = \{(r_1 : \varphi_1), \dots, (r_n : \varphi_n)\}$, where each $r_i \in R$ and φ_i is a formula over propositional variables P , the process of constructing a reward machine (RM) can be outlined as follows:

1. **Transform each formula φ_i into a DFA $A^{(i)}$:** Convert each temporal logic formula φ_i into a DFA that accepts all input sequences satisfying φ_i . This step leverages the equivalence between regular languages and DFAs.
2. **Construct the RM from the combined DFA states:** Define the RM $M_R = \langle Q, q_0, \Sigma, R, \delta, \rho \rangle$ using the combined states from all DFAs. Here:
 - Q is the set of states formed by the Cartesian product of states from $A^{(i)}$.
 - ρ assigns rewards based on the accepting conditions of the DFA states and the transitions between them.

Formally, let $\tau = (s_0, a_0)(s_1, a_1) \dots (s_n, a_n)s_{n+1} \in (S \times A)^+ \times S$ represent the agent's execution history. The sequence of experiences received by the agent along τ is given by $\{(s_i, a_i, s_{i+1})\}_{0 \leq i \leq n}$, where each state overlaps with the next in the sequence.

The projection of τ by the labeling function L is said to entail φ if $L(s_0, a_0, s_1) \dots L(s_n, a_n, s_{n+1})$ satisfies φ . We denote this as $\tau \models_L \varphi$. The reward assigned to τ is defined by:

$$R(\tau) := \sum_{k=1}^n r_k \cdot 1(\tau \models_L \varphi_k), \quad (3.1)$$

where $1(x)$ is the indicator function that evaluates to 1 if x is true and 0 otherwise.

By constructing DFAs for each φ_i and combining them to form the RM, we can design a non-Markovian reward function that assigns rewards based on temporally extended conditions specified by the formulas. This approach ensures that complex, condition-based rewards can be effectively managed in a reinforcement learning framework.

Let $A^{(i)} = \langle Q^{(i)}, \Sigma, q_0^{(i)}, \delta^{(i)}, F^{(i)} \rangle$ be the DFA transformation for each φ_i . We define the reward machine $M_R = \langle Q, q_0, \Sigma, R, \delta, \rho \rangle$ with components as follows:

- $Q := Q^{(1)} \times \dots \times Q^{(n)}$
- $q_0 := (q_0^{(1)}, \dots, q_0^{(n)})$

- $\delta(q, \sigma) := (\delta^{(1)}(q^{(1)}, \sigma), \dots, \delta^{(n)}(q^{(n)}, \sigma))$
- $\rho(q, \sigma) := \sum_{k=1}^n \rho^{(k)}(q^{(k)}, \sigma)$, where $\rho^{(k)}(q, \sigma)(s, a, s') := r_k \cdot 1(\delta^{(k)}(q, \sigma) \in F^{(k)})$.

Here, $1(x)$ denotes the indicator function that evaluates to 1 if x is true and 0 otherwise. Each $\rho^{(k)}(q, \sigma)$ is a constant function, meaning its value does not depend on the specific transition (s, a, s') . Consequently, the space of reward functions R includes constant functions representing the sum of all possible subsets of $\{r_1, \dots, r_n\}$.

3.6 Q-Learning for Reward Machines (QRM)

Q-Learning for Reward Machines (QRM) was introduced as a method to decompose and exploit a reward machine Icarte et al. (2018b). QRM learns one Q-function $\tilde{q}_i(s, a)$ per reward machine (RM) state q_i .

Given an experience (s, a, s') , the RM computes the reward $r_i = \rho(q_i, L(s, a, s'))$ that the agent receives for this experience, assuming it occurred while the RM was in state q_i . The next state in the reward machine is determined as $q_j = \delta(q_i, L(s, a, s'))$. This information facilitates the Q-update for $\tilde{q}_i(s, a)$ as follows:

$$\tilde{q}_i(s, a) \leftarrow \tilde{q}_i(s, a) + \alpha \left[r_i + \gamma \max_{a'} \tilde{q}_j(s', a') - \tilde{q}_i(s, a) \right] \quad (3.2)$$

This update is performed for all $q_i \in Q$ simultaneously, ensuring convergence to an optimal solution in the tabular case.

3.6.1 Augmenting QRM with Reward Shaping

We propose augmenting QRM with reward shaping Ng et al. (1999) to enhance learning efficiency. Reward shaping modifies the reward function to facilitate faster

policy learning without altering the set of optimal policies. The underlying intuition is that certain reward functions are easier for an agent to learn from, even if they yield the same optimal policy as the original reward function.

Ng et al. Ng et al. (1999) demonstrated that given any Markov Decision Process (MDP) $M = \langle S, A, s_0, T, r, \gamma \rangle$ and a potential function $\Phi : S \rightarrow \mathbb{R}$, altering the reward function to

$$r'(s, a, s') = r(s, a, s') + \gamma\Phi(s') - \Phi(s) \quad (3.3)$$

does not change the optimal policy set of the MDP. Therefore, by selecting an appropriate potential function Φ , we can shape the reward landscape to guide the agent towards optimal policies more efficiently while ensuring that these policies remain optimal with respect to the original reward function.

3.7 Reward Shaping with Value Iteration

Formally, given an RM $\langle Q, q_0, \Sigma, R, \delta, \rho \rangle$, we construct an MDP $M = \langle S, A, \emptyset, T, r, \gamma \rangle$, where:

- $S = Q$,
- $A = 2^P$,
- $T(q' \mid q, \sigma) = 1$ if $q' = \delta(q, \sigma)$ (zero otherwise),
- $r(q, \sigma, q') = \rho(q, \sigma)$,
- $\gamma < 1$.

Intuitively, this MDP represents an environment where every transition in the RM corresponds to a deterministic action with the same reward. Notably, M lacks

a designated start state, as the value of being in every state within M must be computed when employing an optimal policy. Specifically, we compute $v^*(q) = \max_{\sigma} q^*(q, \sigma)$, where q^* denotes the optimal Q-function and q is a state in M . This computation is achieved using the well-established value iteration algorithm. Overall process of computing v^* using value iteration, given Q , δ , ρ , and γ , is illustrated in Algorithm 1.

Algorithm 1 Value Iteration for Automatic Reward Shaping

Input: Q, δ, ρ, γ
for all $q_i \in Q$ **do**
 $v(q_i) \leftarrow 0$ ▷ Initialize value function
end for
 $e \leftarrow 1$
while $e > 0$ **do**
 $e \leftarrow 0$
 for all $q_i \in Q$ **do**
 $v_0 \leftarrow \max_{\sigma: \delta(q_i, \sigma) = q_j} [\rho(q_i, \sigma) + \gamma v(q_j)]$
 $e \leftarrow \max(e, |v(q_i) - v_0|)$
 $v(q_i) \leftarrow v_0$
 end for
end while
return v

3.8 Value Iteration for Potential Functions

Value iteration can be applied to the RM to calculate the potential function:

- Construct an MDP where RM states are treated as MDP states.
- Apply value iteration to compute $v^*(q)$, which approximates the cumulative reward for each RM state.
- Define the potential function as $\Phi(s, q) = -v^*(q)$ to encourage transitions toward task completion.

3.9 Example Application: Office World Task

In the office world grid environment, the agent’s task is defined using LTL:

$$\Diamond(\text{coffee} \wedge \Diamond\text{office}) \wedge \Box\neg\text{decoration}$$

The corresponding RM assigns rewards based on achieving sub-goals while avoiding penalties for undesired states, structured to accelerate learning using the potential function derived from value iteration.

Chapter 4

Experimental Results and Observations

4.1 Experimental Setup

To evaluate the effectiveness of reward shaping using value iteration over a Reward Machine (RM), we conducted experiments comparing three distinct approaches:

1. A standard Q-learning agent that takes both the environment state and the current RM state as input.
2. A QRM-based agent as introduced by Toro Icarte et al. Icarte et al. (2018b).
3. A QRM-based agent augmented with reward shaping.

All three approaches were tested across the same three environments, tasks, and RM specifications as those used by Toro Icarte et al. Icarte et al. (2018b). Additionally, we employed the same network architecture and experimental setup when utilizing Deep Reinforcement Learning (Deep RL) methods. The code utilized for these experiments is publicly available at <https://bitbucket.org/RToroIcarte/qrm> Icarte et al. (2018b).

Our evaluation was conducted in a multi-task setting, where each test environment presented the agent with a set of tasks to solve. For all tasks, the agent received a reward of 1 only upon the completion of the task and a reward of 0 at all other steps. During experimentation within an environment, tasks were alternated on an episode-by-episode basis.

The standard Q-learning agent learned a separate Q-function for each task. In contrast, the QRM-based agent exploited the structure of the RMs across different tasks by updating the Q-functions for all states of all RMs, not just those of the current task’s RM. This approach follows the methodology outlined by Toro Icarte et al. Icarte et al. (2018b), where RMs are utilized to simulate state transitions and rewards for experiences (s, a, s') from every state in every RM. Consequently, the Q-function for each of these RM states was updated using Equation (4.1).

$$\tilde{q}_i(s, a) \leftarrow \tilde{q}_i(s, a) + \alpha \left[r_i + \gamma \max_{a'} \tilde{q}_j(s', a') - \tilde{q}_i(s, a) \right] \quad (4.1)$$

4.2 Environments and Tasks

My experiments involved three distinct test environments:

1. **Office World:** This environment comprised four tasks, including delivering coffee and/or mail, and patrolling between several locations. In all tasks, the agent was required to avoid stepping on decorations.
2. **Minecraft Domain:** In this 2D grid world domain, the agent could move in the cardinal directions and pick up different raw materials to build objects. We utilized variants of the ten object-building tasks introduced by Andreas et al. Andreas et al. (2017), removing any unnecessary strict ordering of subtasks.
3. **WaterWorld Domain:** A variant of the continuous WaterWorld domain Karpathy (2015), this environment featured a continuous two-dimensional box

where the agent could move by changing its velocity in one of the four cardinal directions each step. Various colored balls moved around the environment, and the ten tasks involved touching different colored balls, sometimes in a specific sequence. Notably, this variant was fully observable, unlike the original WaterWorld environment.

4.3 Experimental Parameters

The following parameters were consistent across all experiments:

- Discount factor: $\gamma = 0.9$
- Exploration constant: $\epsilon = 0.1$

For the grid world environments (Office World and Minecraft Domain), a step size of $\alpha = 1$ was used. In the WaterWorld domain, a significantly smaller step size of $\alpha = 0.00001$ was employed to accommodate the continuous nature of the environment. Each method was executed 30 times per environment to ensure statistical reliability.

In the grid world environments, tabular RL was utilized, whereas neural networks approximated the Q-functions in the WaterWorld domain to handle the continuous state space.

4.4 Results

The results of our experiments are depicted in Figure 4.1. For each environment, we plotted the average performance over the 30 runs, along with the 25th and 75th percentiles, after normalizing the reward per task using the maximum discounted reward possible for that task. An average value of 1 indicates that the agent has learned to perform optimally on all tasks within that environment across all runs.

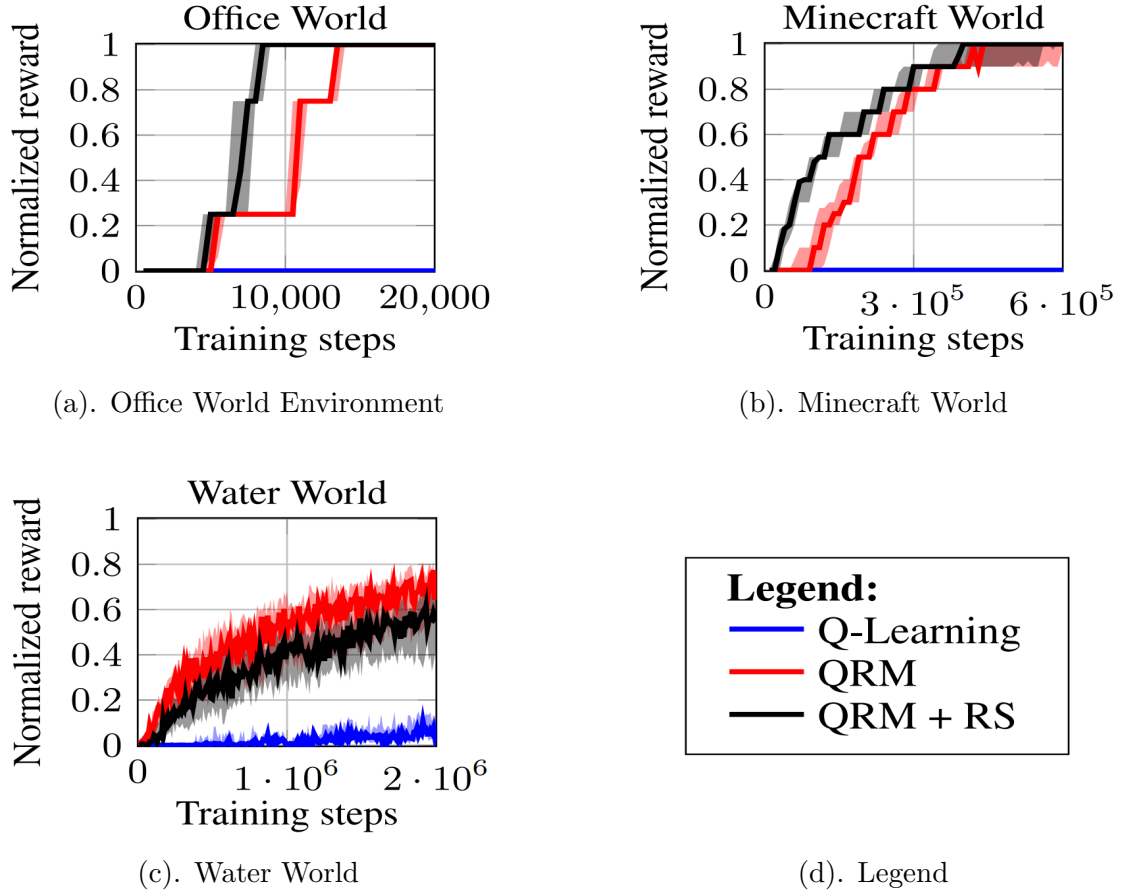


Figure 4.1: Comparison of RL Approaches Across Different Environments. Each subfigure represents the performance of the Q-learning agent, QRM-based agent, and QRM-based agent with Reward Shaping (QRM + RS) in the respective environment. The performance metrics are averaged over 30 runs, with the shaded areas indicating the 25th and 75th percentiles.

4.5 Observations

The experimental results revealed several key insights:

1. **QRM-Based Approaches Outperform Standard Q-Learning:** Consistent with the findings of Toro Icarte et al. Icarte et al. (2018b), QRM-based approaches demonstrated a significant advantage over standard Q-learning across all three environments. This superiority is attributed to the ability of QRM

to exploit the structure of RMs, allowing for more efficient learning across multiple tasks.

2. **Reward Shaping Enhances Performance in Grid World Environments:** The combination of QRM with reward shaping (denoted as "QRM + RS") led to substantial improvements in two of the three domains—the Office World and Minecraft Domain. This enhancement was achieved through a simple, automatic preprocessing step that modified the reward function without compromising the optimality of the learned policies.
3. **Reward Shaping Did Not Improve Performance in WaterWorld Domain:** Interestingly, reward shaping did not yield performance gains in the WaterWorld environment, the only domain where Deep RL was employed. This counter-intuitive result suggests that reward shaping may interact differently with function approximation methods like neural networks, highlighting an area for future research.
4. **Scalability and Efficiency:** The use of value iteration for automatic reward shaping proved effective in discrete grid world environments but faced challenges in continuous environments like WaterWorld, where neural network approximations were necessary. The disparity in step sizes (α) between discrete and continuous domains underscores the need for tailored approaches when scaling reward shaping techniques.
5. **Potential for Future Research:** The lack of improvement in the WaterWorld domain opens avenues for exploring how reward shaping interacts with Deep RL architectures. Additionally, the overall success of QRM-based methods encourages further investigation into learning algorithms that can better exploit the structural information provided by RMs or other automata-based representations.

Chapter 5

Conclusion and Future Work

5.1 Summary of Work

The experiments affirm the efficacy of Reward Machine-based Reward Shaping in enhancing the performance of RL agents in multi-task settings. While QRM-based approaches consistently outperformed standard Q-learning, the integration of reward shaping further boosted performance in discrete environments. The mixed results in continuous environments like WaterWorld highlight the complexity of combining reward shaping with Deep RL, pointing to the need for more nuanced methods in such contexts. Overall, the findings support the potential of automata-based representations in facilitating robust and efficient RL solutions.

5.2 Future Work

5.2.1 Policy Robustness to Environmental Changes

One of the main challenges in RL is ensuring that learned policies remain effective even as the environment undergoes changes. Future work could explore:

- **Adaptive Policy Learning:** Developing RL algorithms that incorporate meta-learning or transfer learning to adapt policies when environmental dynamics change. This would enable agents to modify their behavior without requiring complete retraining.
- **Environment Monitoring and Model Updating:** Integrating real-time monitoring tools that detect changes in the environment and update the model of the environment accordingly. This can be done using Bayesian inference or online learning techniques.
- **Robustness Through Generalization:** Training policies that are generalized across a range of environmental conditions to improve adaptability and reduce performance drops in unseen scenarios.

5.2.2 Balancing Safety Constraints: Flexibility

Another crucial aspect of RL in environments with safety considerations is determining the appropriate level of strictness for safety constraints. Future research could focus on:

- **Dynamic Safety Levels:** Creating a flexible framework where the strictness of safety constraints can be adjusted based on task criticality and environmental complexity.
- **Adaptive Safety Constraints:** Implementing a mechanism that allows the agent to assess when it can temporarily relax safety constraints for higher reward and when it should adhere strictly to safety protocols.
- **Trade-off Analysis:** Studying the trade-offs between harsh and lenient safety policies to understand their impact on learning efficiency and task performance.

5.2.3 Scope for Future Work: Quadrant Analysis

To illustrate future research directions, we propose a four-quadrant analysis with the type of environment on the x-axis (from static to dynamic) and the nature of safety constraints on the y-axis (from static to dynamic). Each quadrant represents a unique combination and potential area of focus:

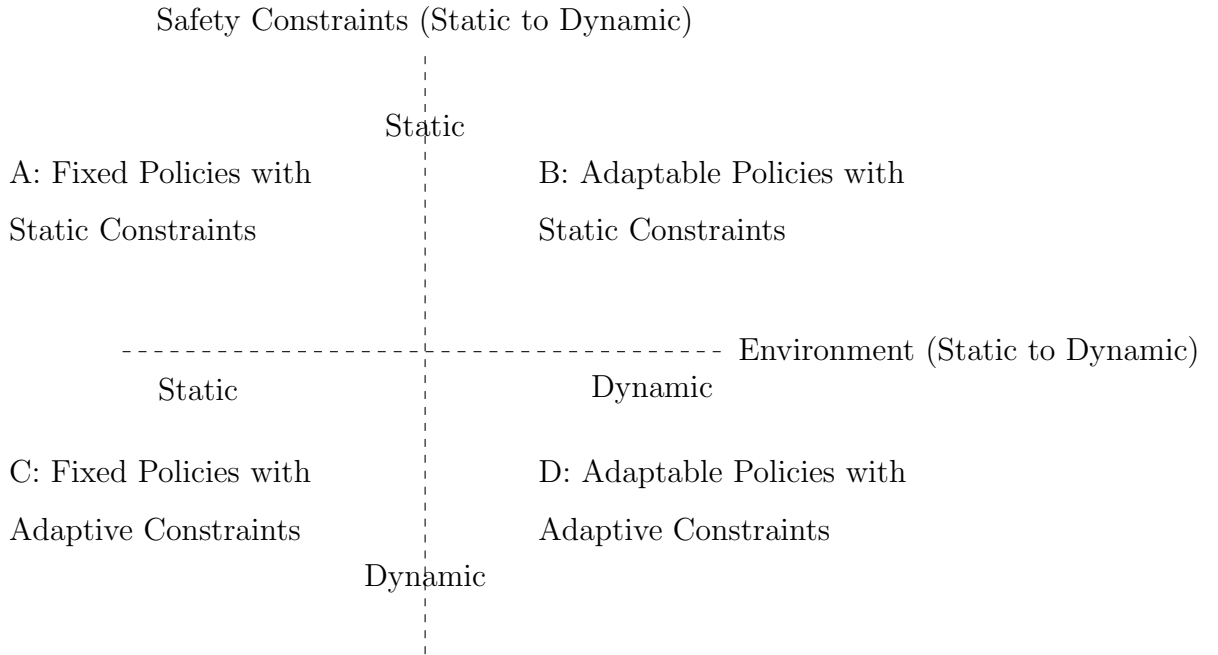


Figure 5.1: Quadrant Analysis for Future Work in RL

- **Quadrant A (Static Environment, Static Safety Constraints):** This area focuses on developing robust policies that operate under fixed safety rules in environments that do not change. The research here ensures that agents maintain consistent performance and safety adherence without any adaptability requirements.
- **Quadrant B (Dynamic Environment, Static Safety Constraints):** This quadrant emphasizes policies that adapt to changing environments while operating under fixed safety constraints. Research in this area explores how agents can respond to new situations while still complying with predefined safety rules.

- **Quadrant C (Static Environment, Dynamic Safety Constraints):** Investigates policies where the safety constraints change over time, but the environment remains static. This area studies how agents can adjust their behavior to accommodate evolving safety requirements in a stable environment.
- **Quadrant D (Dynamic Environment, Dynamic Safety Constraints):** This quadrant represents the most complex scenario, where both the environment and safety constraints change dynamically. Future work here involves designing policies that can adapt to both types of changes, ensuring optimal task performance and flexible adherence to safety protocols.

5.2.4 Potential Research Directions

Research in these quadrants can lead to:

- **Dynamic Adaptation Strategies:** Methods that enable agents to update their policies in real-time in response to changing environments and safety constraints.
- **Balancing Trade-offs:** Studying the trade-offs between adaptability and computational complexity.
- **Safety-Aware Meta-Learning:** Techniques that empower agents to learn how to adjust their policies and safety protocols autonomously.
- **Real-World Implementation:** Applying these findings in real-world systems like autonomous vehicles and industrial robots to test their viability and effectiveness.

This quadrant analysis offers a structured roadmap for advancing reinforcement learning to better handle dynamic environments and adaptive safety constraints.

Bibliography

- Andreas, J., Klein, D., and Levine, S. (2017). Modular multitask reinforcement learning with policy sketches. In *ICML*, pages 166–175.
- Bacchus, F., Boutilier, C., and Grove, A. J. (1996). Rewarding behaviors. In *AAAI*, pages 1160–1167.
- Duret-Lutz, A., Lewkowicz, A., Fauchille, A., Michaud, T., Renault, E., and Xu, L. (2016). Spot 2.0 — a framework for ltl and -automata manipulation. In *ATVA*, volume 9938 of *LNCS*, pages 122–129. Springer.
- Emerson, E. A. (1990). Temporal and modal logic. In *Handbook of Theoretical Computer Science*, volume B, pages 995–1072. Elsevier.
- Hasselt, H. V., Guez, A., and Silver, D. (2016). Deep reinforcement learning with double q-learning. In *AAAI*, pages 2094–2100.
- Hopcroft, J. E. and Ullman, J. D. (1979). *Introduction to Automata Theory, Languages and Computation*. Addison-Wesley.
- Icarte, R. T., Klassen, T. Q., Valenzano, R. A., and McIlraith, S. A. (2018a). Teaching multiple tasks to an rl agent using ltl. In *AAMAS*, pages 452–461.
- Icarte, R. T., Klassen, T. Q., Valenzano, R. A., and McIlraith, S. A. (2018b). Using reward machines for high-level task specification and decomposition in reinforcement learning. In *ICML*, pages 2112–2121.

- Karpathy, A. (2015). Reinforcejs: Waterworld demo. <http://cs.stanford.edu/people/karpathy/reinforcejs/waterworld.html>.
- Lu, T., Schuurmans, D., and Boutilier, C. (2018). Non-delusional q-learning and value-iteration. In *NeurIPS*, pages 9971–9981.
- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533.
- Ng, A. Y., Harada, D., and Russell, S. (1999). Policy invariance under reward transformations: Theory and application to reward shaping. In *ICML*, volume 3, pages 278–287.
- Schaul, T., Quan, J., Antonoglou, I., and Silver, D. (2015). Prioritized experience replay. *arXiv preprint arXiv:1511.05952*.
- Sutton, R. S. and Barto, A. G. (2018). *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition.
- Thompson, K. (1968). Regular expression search algorithm. *Communications of the ACM*, 11(6):419–422.
- Watkins, C. J. C. H. and Dayan, P. (1992). Q-learning. *Machine Learning*, 8(3-4):279–292.